K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

AI in Cyber Security Lab

[ENSP355]

Lab Assignment 01 (Unit 1): Building an AI-Driven Cyber Threat Awareness and Detection Prototype

KR. MANGALAM UNIVERSITY For BACHELORS IN TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING With Specialisation In AI & ML

SUBMITTED BY:

Meenakshi
2301730200
Btech Cse AI & ML

SUBMITTED TO:

Dr. Monika Khatkar

GitHub Repository: https://github.com/meenakship0805/AI-inCyberSecurity-Assignment-Meenakshi-2301730200

1. Problem Overview

Every organisation today depends on people logging in to its systems, and the login page is usually the first place an attacker knocks. Rule-based tools such as "block after 5 failed attempts" catch the obvious cases, but attackers adapt: they spread guesses across many IP addresses, or they use a stolen but valid account so nothing looks broken.

In this project I take the role of an entry-level security analyst. The aim is to build a small prototype that looks at login events and labels each one as **normal** or **suspicious**, and then to honestly check where such a system can go wrong for real users.

The work covers five parts: environment setup, threat analysis of the data, classic machine learning models, a small neural network, and an ethics study on false positives and bias.

2. Methodology

2.1 Environment (Task 1)

A public GitHub repository was created with *data/*, *src/*, *outputs/* and *report/* folders, a README, a .gitignore and a requirements.txt. A Python virtual environment was set up with numpy, pandas, scikit-learn, matplotlib, seaborn and jupyter. The script *check_setup.py* prints all library versions and trains a tiny model to prove the setup works. The full project runs end to end with one command: *python src/run_all.py*.

2.2 Dataset

Real login logs from companies are confidential, so I wrote a simulator (*generate_data.py*) that produces 5,000 login events, 20% of them suspicious. The suspicious events follow three well-documented attack patterns from MITRE ATT&CK:

Attack type	Events	How it shows up in the data
Brute force (T1110)	420	Many failed logins, night hours, password only
Credential stuffing	380	Many distinct IPs in 24h, frequent VPN use
Insider exfiltration (T1567)	200	Valid user, very large downloads, long late sessions

Each event has 10 features: user role, device type, location, auth method, hour of day, failed attempts in the last hour, session length, data downloaded, distinct IPs in 24 hours and a VPN flag. To keep the data realistic, about 3% of values in four columns were removed and 4% of rows were given noisy failed-attempt counts so the two classes overlap. One limitation should be stated clearly: because the data is simulated, the scores below are higher than they would be on real logs.

2.3 Preprocessing

- **Missing values:** median for numbers (robust to big outliers like download size), most frequent value for categories.
- **Encoding:** one-hot encoding for the four categorical columns.
- **Scaling:** StandardScaler on numeric columns, which matters for Logistic Regression and the neural network.
- **Split:** stratified 75/25 train/test split (3,750 / 1,250 rows) so both sets keep the same 20% attack rate.
- All steps live inside one scikit-learn Pipeline, so every model sees exactly the same data and nothing from the test set leaks into training.

2.4 Models

- **Logistic Regression** - simple linear baseline, easy to explain.
- **Decision Tree** - depth limited to 6 and at least 10 samples per leaf to avoid memorising the training data.
- **Neural Network (MLPClassifier)** - two hidden layers (32 and 16 neurons), ReLU, Adam optimiser, early stopping on a validation split.

Models were judged on accuracy, precision, recall and F1. Because only 20% of events are attacks, accuracy alone is misleading: a model that calls everything normal would still score 80%. Recall (attacks caught) and precision (alerts that are real) tell the actual story.

3. Results and Observations

3.1 Threat landscape analysis (Task 2)

Feature	Normal (avg)	Suspicious (avg)	Ratio
Failed attempts in last hour	0.55	4.85	8.8x
Data downloaded (MB)	30.5	136.7	4.5x
Distinct IPs in 24h	2.2	4.8	2.2x
VPN use	19%	36%	1.9x
Session duration (min)	45.4	26.5	0.6x

- **Failed login attempts are the strongest single warning sign** - nearly 9 times higher for suspicious events.
- Average hour of day is almost the same for both classes (13.2 vs 13.4), which looks like it carries no signal. The hourly plot shows why this is wrong: attacks cluster at night (0-5h and 20-23h) while normal logins peak at midday. An average hides a two-peak pattern.
- Download size separates insider theft clearly but only on a log scale; most attacks look normal on this feature, so it cannot work alone.
- 88% of 'foreign' logins are suspicious. This looks like a useful feature but it becomes the main ethical problem in Task 5.

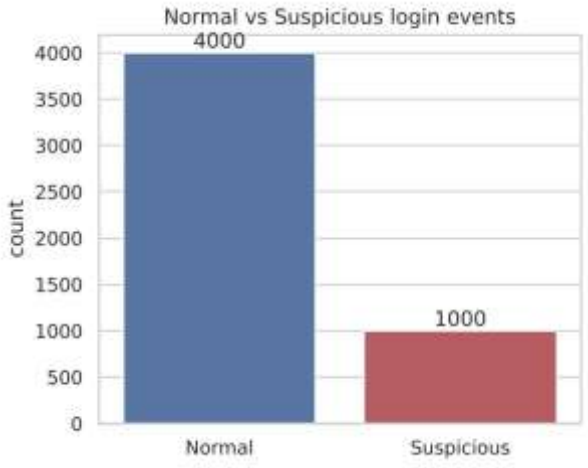


Figure 1: Class balance - 4,000 normal vs 1,000 suspicious events

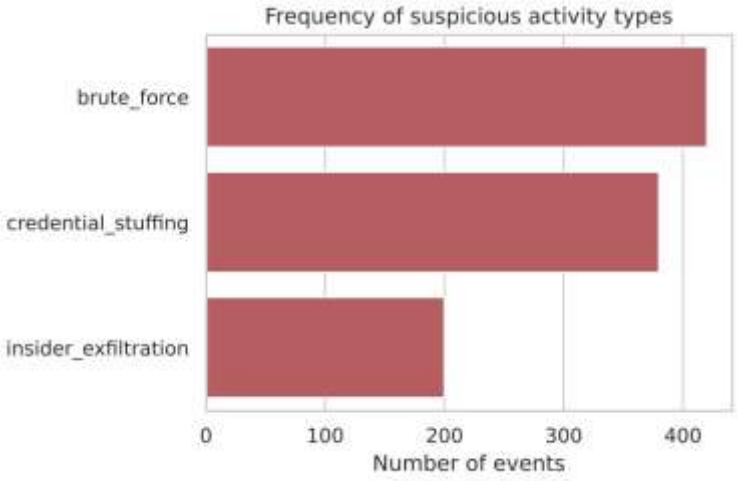


Figure 2: Frequency of each suspicious activity type

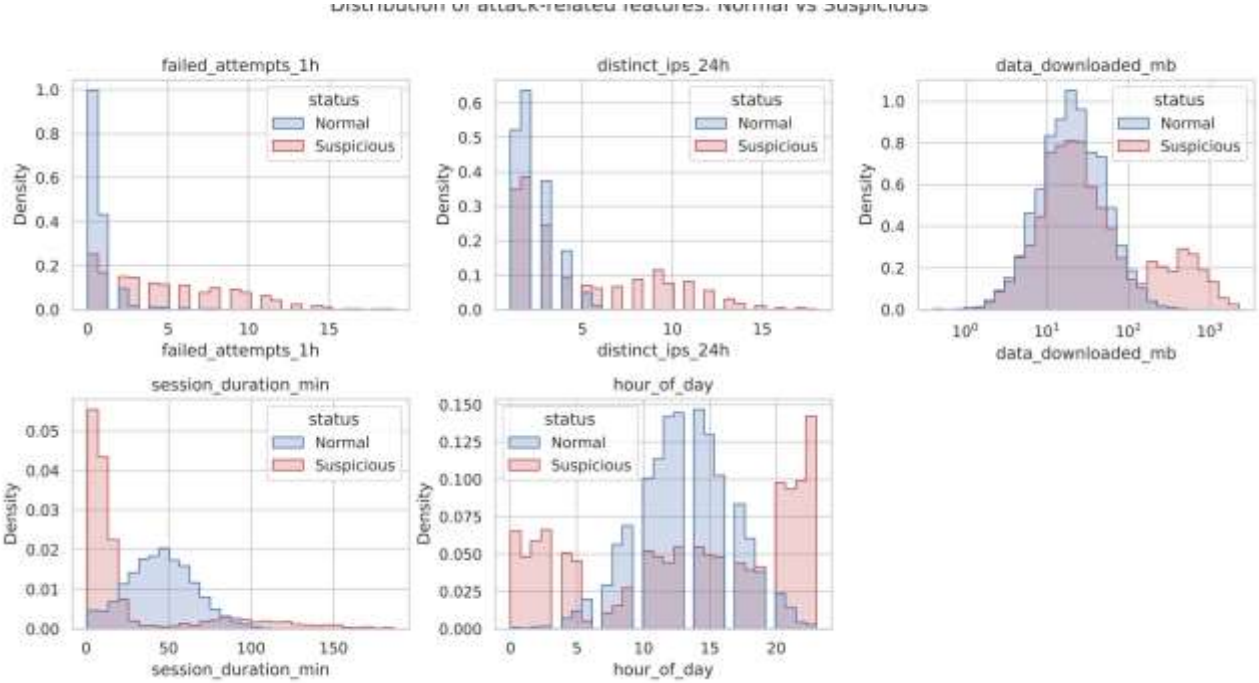


Figure 3: Distribution of attack-related features, normal vs suspicious

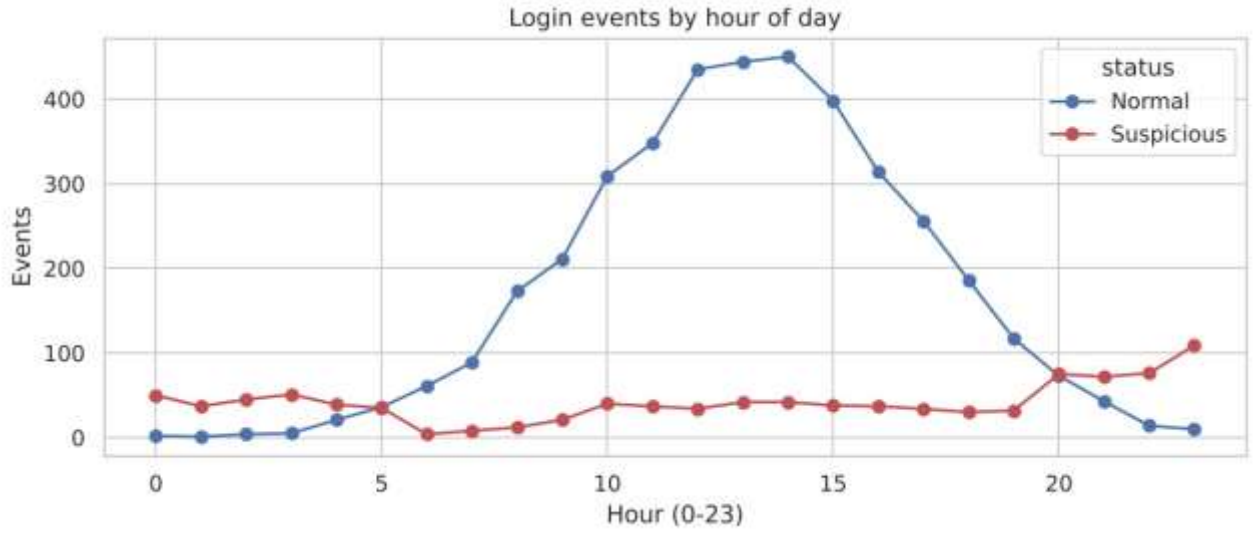


Figure 4: Login events by hour - attacks peak at night

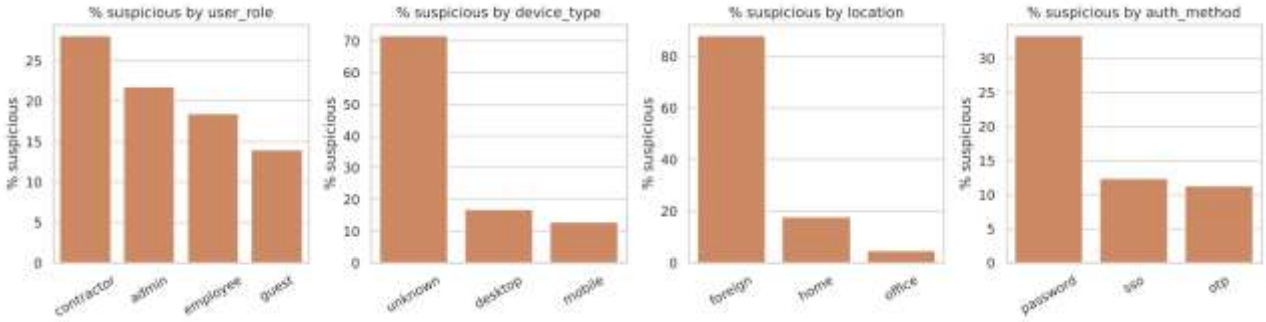


Figure 5: Share of suspicious events in each category

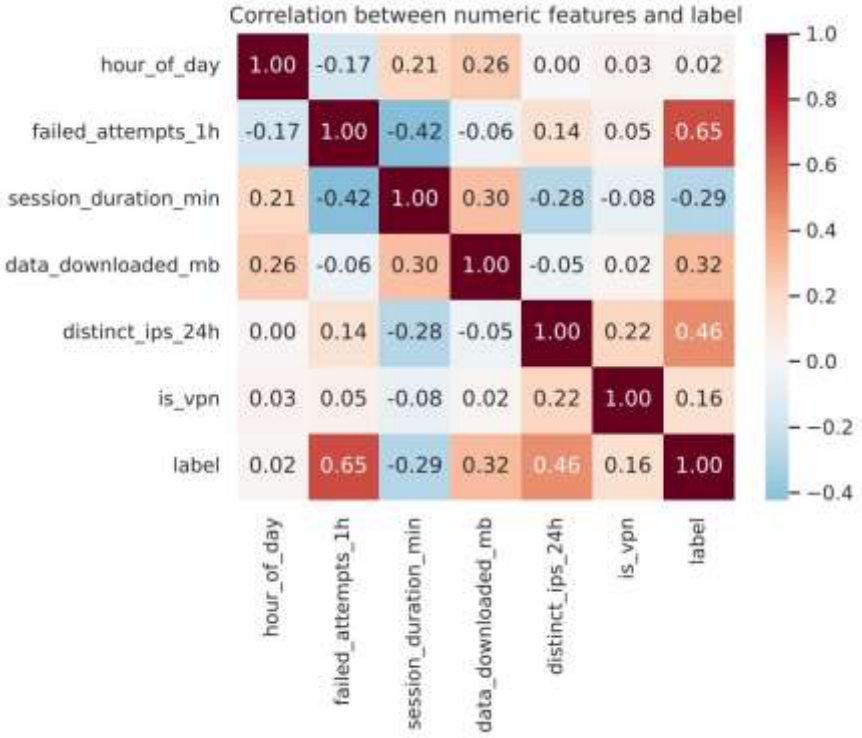


Figure 6: Correlation of numeric features with the label

3.2 Machine learning classification (Task 3)

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.978	0.955	0.932	0.943
Decision Tree	0.977	0.987	0.896	0.939

- Both models have almost the same accuracy, but they fail in opposite ways. The Decision Tree raises very few false alarms (precision 0.987) but misses 26 of 250 attacks. Logistic Regression catches more attacks (17 missed) at the cost of more false alarms.
- For a security team, missing an attack is usually worse than an extra alert, so Logistic Regression is the better choice of the two here.
- The tree relies mostly on failed attempts (49% importance), distinct IPs (19%) and data downloaded (18%) - the same signals found in the EDA, which is a good sign the model learned real behaviour.

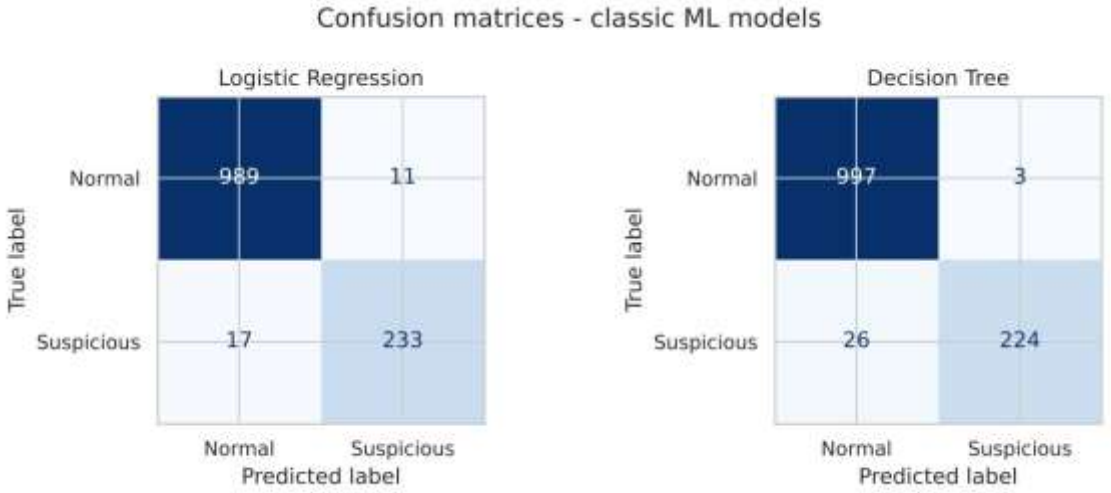


Figure 7: Confusion matrices for Logistic Regression and Decision Tree

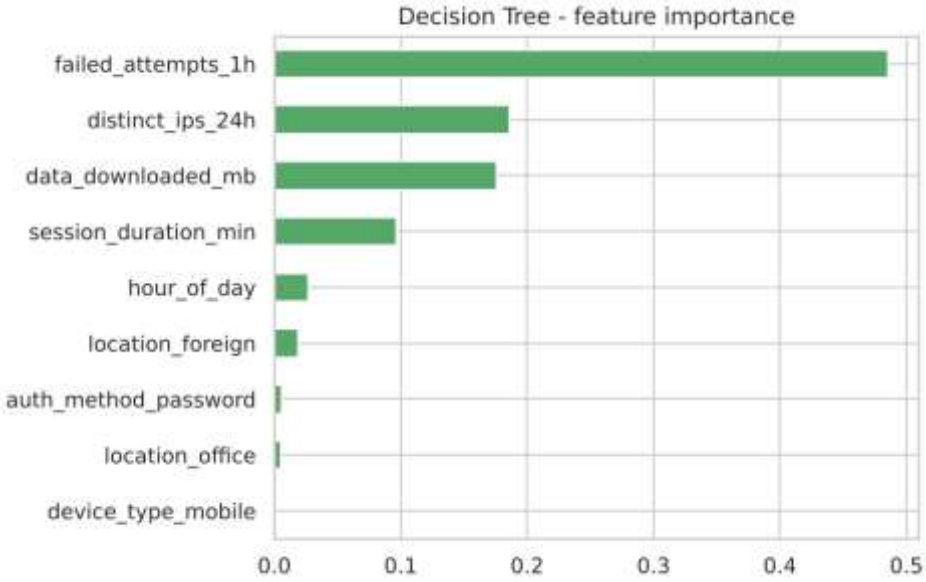


Figure 8: Decision Tree feature importance

3.3 Neural network and ML vs DL comparison (Task 4)

Model	Accuracy	Recall	F1	Train time	Predict (per 1k)
Logistic Regression	0.978	0.932	0.943	0.027 s	5.7 ms
Decision Tree	0.977	0.896	0.939	0.024 s	5.5 ms
Neural Network (MLP)	0.989	0.960	0.972	0.375 s	6.6 ms

- **Detection accuracy:** the MLP is the best on every metric. It misses only 10 attacks against 17 for Logistic Regression, because it can combine signals (for example "late hour AND large download AND contractor") that a linear model cannot.
- **Training time:** the MLP takes about 14x longer to train. In absolute terms it is still under half a second, because the dataset is small. On millions of real log events this gap would matter a lot more. Prediction speed is about the same for all three.
- **Practical feasibility:** the gain is real but small (F1 0.943 to 0.972) on simulated data, and the MLP is a black box. An analyst cannot easily answer "why was this user blocked?". A sensible setup would be Logistic Regression or a tree as the main, explainable detector, with the MLP as a second opinion for borderline cases.
- Early stopping ended training after 30 epochs, once validation accuracy stopped improving, which keeps the network from overfitting.

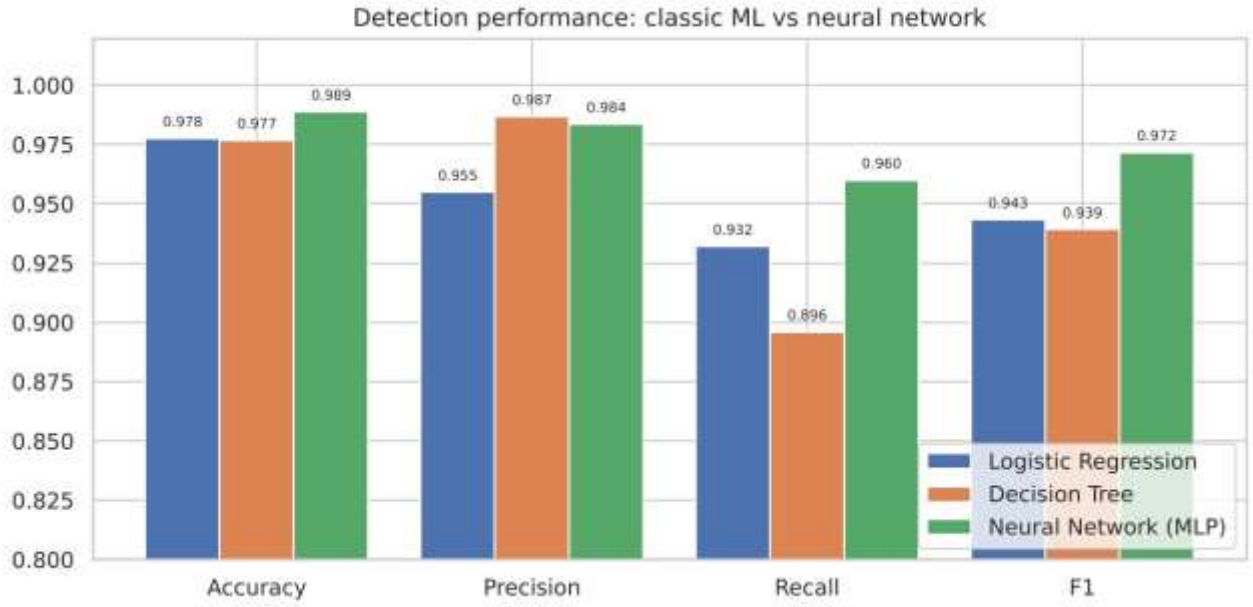


Figure 9: Detection metrics for all three models



Figure 10: Average training time over 3 runs

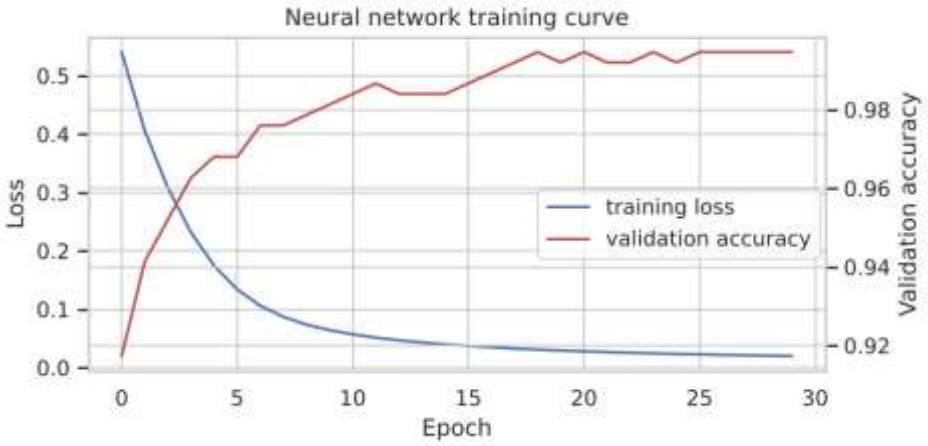


Figure 11: Neural network loss and validation accuracy per epoch

4. Ethical Discussion (Task 5)

4.1 Experiment A - false positives block legitimate users

Logistic Regression outputs a probability. By default a login is flagged if that probability is above 0.5. I moved this threshold from 0.05 to 0.95 and counted the results on the 1,250 test events (1,000 genuine, 250 attacks):

Threshold	Genuine users blocked	Attacks missed	Recall
0.1 (very strict)	55	7	0.972
0.3	18	9	0.964
0.5 (default)	11	17	0.932
0.7	6	27	0.892
0.9 (very relaxed)	2	50	0.800

- Going from 0.5 to 0.1 catches 10 more attacks but blocks 5 times as many genuine users (11 to 55). Scaled to a company with 100,000 logins a day, that is thousands of locked-out employees and a flooded help desk.
- Going the other way, a relaxed threshold of 0.9 lets 1 in 5 attacks through.
- There is no "correct" threshold. It is a business and ethical choice about who carries the cost of mistakes. A practical answer is to not block outright in the grey zone (say 0.3 to 0.7) but ask for an extra check like OTP instead.

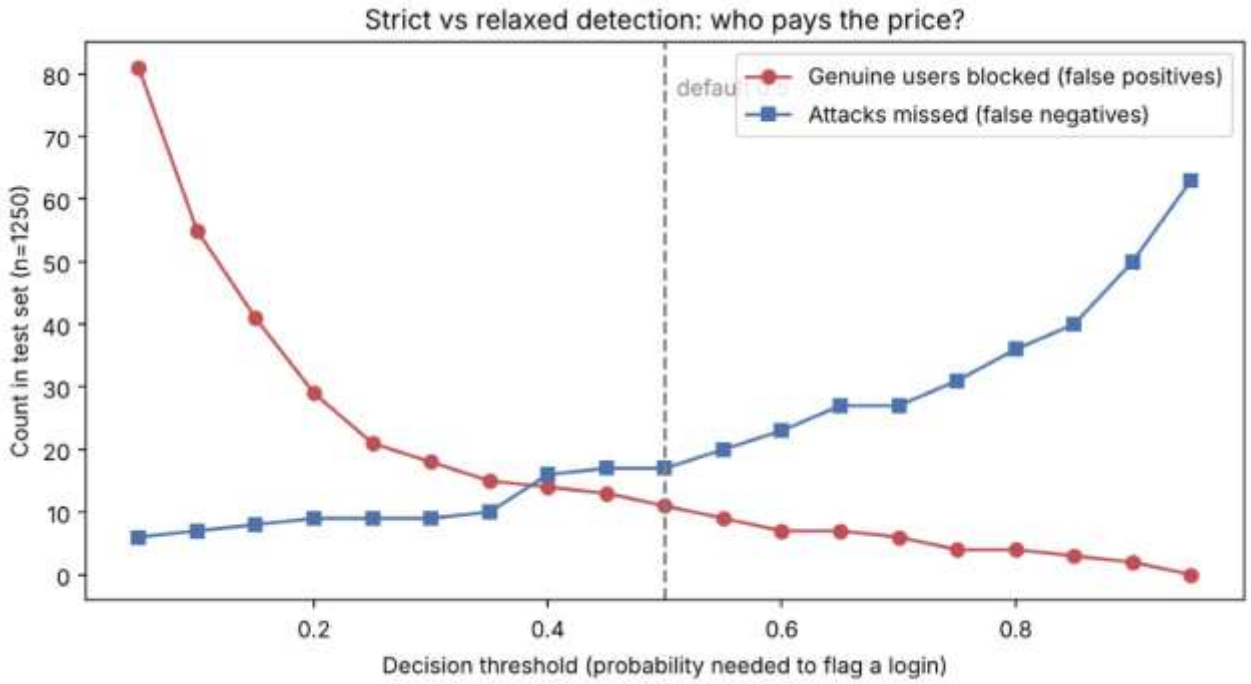


Figure 12: Effect of the decision threshold on blocked users and missed attacks

4.2 Experiment B - biased data leads to unfair decisions

In the training data, 87% of logins from abroad were attacks, simply because genuine staff rarely travel. The model therefore learns that "foreign" itself means danger. To test this I created 300 **genuine** employees working from abroad whose behaviour is completely normal - only their location is different.

Model	Office/home users wrongly blocked	Genuine travellers wrongly blocked	Recall
With location feature	0.9%	10.0% (30 of 300)	0.932
Location feature removed	1.4%	0.7% (2 of 300)	0.880

- With the location feature, a genuine employee abroad is **11 times more likely** to be blocked than one at the office, for exactly the same behaviour.
- Removing the feature almost removes the unfairness (10% to 0.7%) but recall drops from 0.932 to 0.880, so a few more real attacks get through. Fairness here has a measurable cost and someone has to decide it is worth paying.

- Removing one column is not a full fix. Other features such as VPN use or distinct IPs can act as a stand-in for location. Better options are collecting more genuine travel data, checking error rates per group before deployment, and using step-up verification instead of a hard block.

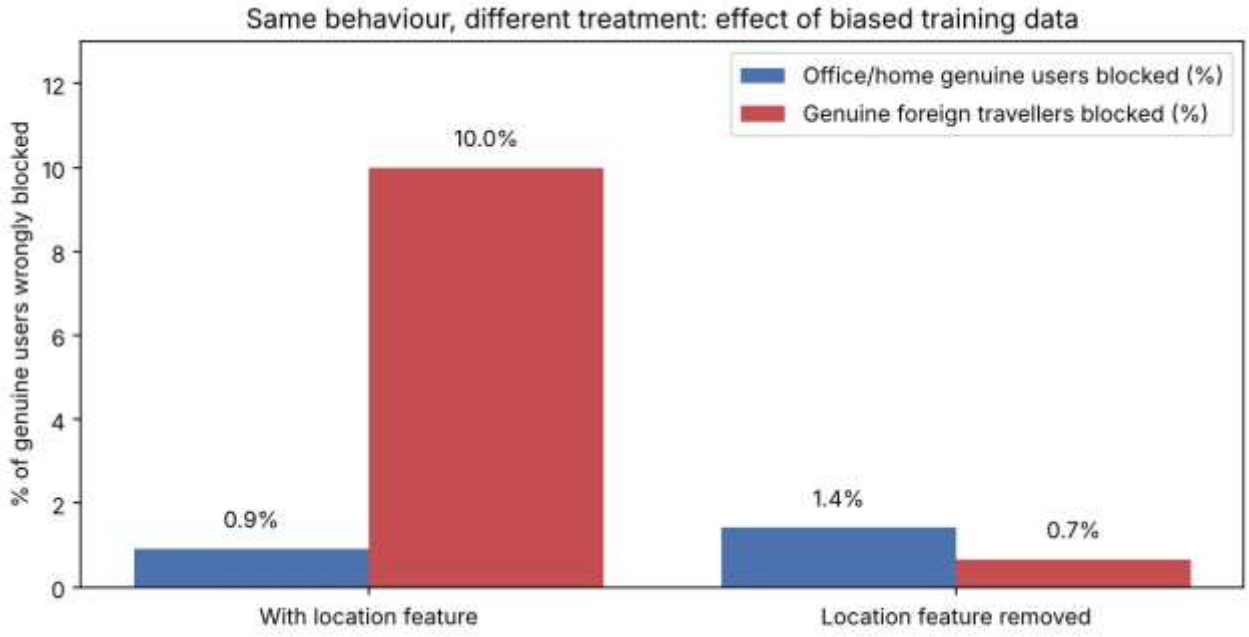


Figure 13: Wrongful blocks for office/home users vs genuine travellers

4.3 Ethical risks

- **Unfair treatment:** groups that are rare in the history (remote staff, travellers, night-shift workers) get flagged more, as shown above.
- **Over-trust in the model:** 98% accuracy sounds safe, but at scale even 1% wrong blocks is a large number of real people.
- **Attackers adapt:** once attackers learn the model watches failed attempts, they slow down. The model needs regular retraining and monitoring for drift.
- **Automation without appeal:** if the model blocks accounts automatically with no quick way to appeal, employees lose access to their work for decisions nobody can explain.

4.4 Privacy concerns

- Login monitoring needs location, IP address, device and working hours. Together these build a detailed picture of each person's daily life.
- Only the fields needed for security should be collected, kept for a fixed period, and access-controlled. Under India's DPDP Act 2023, employees should be told what is collected and why.
- Data downloaded and session length can reveal what people work on. These logs must not be reused for performance tracking without consent.

4.5 Explainability limitations

- Logistic Regression and the Decision Tree can show which features drove a decision, so an analyst can explain a block to the user.
- The neural network gives the best scores but no simple reason for each decision. In security, an alert that cannot be explained is hard to act on, hard to audit and hard to defend if it was wrong.
- Tools like SHAP or LIME could add explanations to the MLP, but they are approximations and add their own complexity.

5. Conclusion

The prototype shows that machine learning can spot suspicious logins well: all three models reached F1 above 0.93, with the neural network best at 0.972. The more important lessons came from Task 5. The threshold decides who pays for mistakes, and skewed history can make a model treat honest users unfairly based on where they are. For a real deployment I would choose an explainable model, use step-up verification instead of hard blocks in uncertain cases, and check error rates for each user group before going live.

Limitations: the data is simulated, so real-world scores would be lower; only one random split was used, so small differences between models (for example 0.977 vs 0.978) are not meaningful.

6. References

- scikit-learn developers. scikit-learn: Machine Learning in Python - Documentation. https://scikit-learn.org/stable/
- NIST. The NIST Cybersecurity Framework (CSF) 2.0, 2024. https://www.nist.gov/cyberframework
- MITRE ATT&CK. T1110 Brute Force; T1078 Valid Accounts; T1567 Exfiltration Over Web Service. https://attack.mitre.org/
- Government of India. Digital Personal Data Protection Act, 2023.
- Dataset: synthetic, generated by the author using src/generate_data.py (included in the repository).